This notebook was created by Jean de Dieu Nyandwi for the love of machine learning community. For any feedback, errors or suggestion, he can be reached on email (johnjw7084 at gmail dot com), Twitter, or LinkedIn.

Neural Networks for Classification with TensorFlow

Contents

Intro to Classification with TensorFlow

Neural networks can also be used for classification problems. In classification, we are mainly predicting the class or categories.

There are two three types of classification problems:

- **Binary classification**: For this classification type, we have two classes. Example might be to classify a given tweet as positive or negative depending on its content.

In binary classification, you only need a single output neuron with a logistic activation function(or sigmoid) that output a number between 0 and 1. A threshold value (by default, 0.5) can be used to differentiate positive and negative classes. Take an example, if the value of the

output neuron is 0.7 (which is greater than 0.5), the tweet can be predicted as positive. Else if the the ouput is 0.4(less than 0.5), the predicted class is negative.

The common loss/cost function used in binary classification is `binary cross entropy`.

- **Multilabel binary classification**: A good example of this classification type is if you wanted to classify a tweet as sarcastic or not, but also simultaneously predict if its content is techy or not. This is just an example.
  Same as binary classification, the output neurons won't be 1, but it will have a logistic activation function(`sigmoid`). In the example above, the output neurons will be two. One neuron for sarcastic/not, and other for techy/not. While the sum of the probability of positive and negative classes will be 1 in binary classification, in multilabel classification, the outputs won't necessarily add up to 1 because we have more than two neurons. In the given example, the sum of the output neuron's values will be 2.

The common loss/cost function used in multilabel binary classification is `binary cross entropy`.

- **Multiclass classification**: For this classification type, we have more than two classes.

The output neurons are equivalent to the number of classes. Take an example. If we are building a system that can classify 10 different fashions, we will have 10 output neurons activated by the `softmax` function. By using `softmax` function, the output will be a vector whose dimension is equivalent to the number of classes. And one thing I like about softmax, is that instead of getting the probabilities in such output vector, we get an an actual classes (0 or 1). Take an example: if the predicted fashion is a bag, and its position in classes is 3, the output will look like this: [0,0,0,1,0,0,0,0,0,0]. In simple words, the predicted class will be 1, and everything else will be 0.

The common loss/cost function used in multiclass classification is `categorical cross entropy`.

In both of those classification types, the number of input neurons, the activation functions in the hidden layers, and the number of hidden layers depend on the problem you're solving.

Below is a summary of hyerparameter best practices in neural network classifiers.

Hyperparameter	Binary classifier	Multilabel binary classifier	Multiclass classifier
Neurons at input layer	depend on the problem	depend on the problem	depend on the problem
No of hidden layer(s)	depend on problem, usually from 1-10	Same as ◁	Same as ◁
Neurons per hidden layer	depend on problem, usually 10-100	Same as ◁	Same as ◁
Neurons at output layer	1	Neurons equivalent to number of labels	Neurons equivalent to number of classes
Activation in hidden layers	Mostly Relu or its variants(LeakyReLU, SeLU	Same as ◁	Same as ◁
Activation in output	sigmoid	sigmoid	softmax

Hyperparameter	Binary classifier	Multilabel binary classifier	Multiclass classifier
layer			
Loss function	binary cross entropy	binary cross entropy	categorical cross entropy
Optimizer	Mostly: SGD, Adam, RMSProp	Same as ◁	Same as ◁ Same as ◁

Table: Typical values of hyperparameters in neural network classifiers

There are many hyperparameters in neural networks and finding the best values of each and each can be overwhelming.

In later notebooks, we will use Keras Tuner to search the best hyperparameters whenever possible. It is nearly impossible to assume that a given value of hyperparamater will work well at first. We usually have to experiment with different values.

Let's put all of the above into practice.

2. Getting Started: Binary Classifier

We will first practice building neural networks for binary classifier. In binary classification, we have two classes.

We will use a classical cancer dataset to predict if a given patient has a malignant or benign based on their medical information. We will get it from sklearn datasets. You can read more about the dataset here.

The dataset contains two labels: malignant, benign.

2.1 Getting the data

We will get the data from sklearn datasets.

```
import numpy as np
import pandas as pd
import sklearn
import seaborn as sns
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras

from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
```

```
# the dataset contain the following features

list(data.feature_names)
```

```
['mean radius',
 'mean texture',
 'mean perimeter',
 'mean area',
 'mean smoothness',
 'mean compactness',
 'mean concavity',
 'mean concave points',
 'mean symmetry',
 'mean fractal dimension',
 'radius error',
 'texture error',
 'perimeter error',
 'area error',
 'smoothness error',
 'compactness error',
 'concavity error',
 'concave points error',
 'symmetry error',
 'fractal dimension error',
 'worst radius',
 'worst texture',
 'worst perimeter',
 'worst area',
 'worst smoothness',
 'worst compactness',
 'worst concavity',
 'worst concave points',
 'worst symmetry',
 'worst fractal dimension']
```

```
# the dataset contain the following labels

data.target_names
```

```
array(['malignant', 'benign'], dtype='<U9')
```

```
# Getting features and labels

X = data.data
y = data.target

# the features and labels are numpy array

type(X)
```

```
numpy.ndarray
```

```
# To quickly look in data we can get the dataframe from X

data_df = pd.DataFrame(X, columns=data.feature_names)
```

2.2 Taking a look in the data

```
# Looking from the head

data_df.head()
```

```
   mean radius  mean texture  ...  worst symmetry  worst fractal
dimension
0        17.99         10.38  ...          0.4601
0.11890
1        20.57         17.77  ...          0.2750
0.08902
2        19.69         21.25  ...          0.3613
0.08758
3        11.42         20.38  ...          0.6638
0.17300
4        20.29         14.34  ...          0.2364
0.07678

[5 rows x 30 columns]
```

```
# Getting the basic information

data_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
```

```
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
dtypes: float64(30)
memory usage: 133.5 KB
```

```
# Getting the basic stats

data_df.describe().transpose()
```

	count	mean	...	75%	max
mean radius	569.0	14.127292	...	15.780000	28.11000
mean texture	569.0	19.289649	...	21.800000	39.28000
mean perimeter	569.0	91.969033	...	104.100000	188.50000
mean area	569.0	654.889104	...	782.700000	2501.00000
mean smoothness	569.0	0.096360	...	0.105300	0.16340
mean compactness	569.0	0.104341	...	0.130400	0.34540
mean concavity	569.0	0.088799	...	0.130700	0.42680
mean concave points	569.0	0.048919	...	0.074000	0.20120
mean symmetry	569.0	0.181162	...	0.195700	0.30400
mean fractal dimension	569.0	0.062798	...	0.066120	0.09744
radius error	569.0	0.405172	...	0.478900	2.87300
texture error	569.0	1.216853	...	1.474000	4.88500
perimeter error	569.0	2.866059	...	3.357000	

```
21.98000
area error                569.0   40.337079  ...    45.190000
542.20000
smoothness error          569.0    0.007041  ...     0.008146
0.03113
compactness error         569.0    0.025478  ...     0.032450
0.13540
concavity error           569.0    0.031894  ...     0.042050
0.39600
concave points error      569.0    0.011796  ...     0.014710
0.05279
symmetry error            569.0    0.020542  ...     0.023480
0.07895
fractal dimension error   569.0    0.003795  ...     0.004558
0.02984
worst radius              569.0   16.269190  ...    18.790000
36.04000
worst texture             569.0   25.677223  ...    29.720000
49.54000
worst perimeter           569.0  107.261213  ...   125.400000
251.20000
worst area                569.0  880.583128  ...  1084.000000
4254.00000
worst smoothness          569.0    0.132369  ...     0.146000
0.22260
worst compactness         569.0    0.254265  ...     0.339100
1.05800
worst concavity           569.0    0.272188  ...     0.382900
1.25200
worst concave points      569.0    0.114606  ...     0.161400
0.29100
worst symmetry            569.0    0.290076  ...     0.317900
0.66380
worst fractal dimension   569.0    0.083946  ...     0.092080
0.20750

[30 rows x 8 columns]
```

2.3 Preparing the Data

The data from sklearn is reasonably cleaned. Let's split the data into train and test sets, and we will follow with scaling the values to be between 0 and 1.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y,
test_size=0.2, shuffle=True, random_state=42)
```

After splitting the data into training and testing sets, let's see the number of examples in each set.

```
print('The number of training samples: {}\nThe number of testing samples: {}'.format(X_train.shape[0], X_test.shape[0]))
```

```
The number of training samples: 455
The number of testing samples: 114
```

```
# Scaling the features to be between 0 and 1.

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)
```

Let's also scale the test set. We do not fit the scaler on the test set. We only transform it.

```
X_test_scaled = scaler.transform(X_test)
```

We are now ready to create, compile and train the model.

2.4 Creating, Compiling and Training a Model

In TensorFlow, creating a model is only putting together an empty graphs. We are going to use Sequential API to stack the layers, from the input to output.

In model compilaton, it's where we specify the `optimizer` and `loss` function. Loss function is there for calculating the difference between the predictions and the actual output, and optimizer is there for reducing the loss.

Also, if we are interested in tracking other metrics during training, we can specify them in `metric`.

```
# Creating a model

# Getting the input shape

input_shape = X_train_scaled.shape[1:]

model_1 = tf.keras.models.Sequential([

              # The first layer has 30 neurons(or units)

              tf.keras.layers.Dense(units=30, input_shape=input_shape, activation='relu'),
```

```
            # The second layer has 25 neurons

            tf.keras.layers.Dense(units=15, activation='relu'),

            # The third layer has 1 neuron and activation of
sigmoid.
            # Because of sigmoid, the output of this layer will be a
value bwteen 0 and 1
            tf.keras.layers.Dense(1, activation='sigmoid')

])

# Compiling the model

model_1.compile(optimizer='sgd',
            loss='binary_crossentropy',
             metrics=['accuracy'])
```

After the model is created and compiled, it's time to teach it. It's time to train it on the data.

```
# By setting validation_split=0.15, I am allocating 15% of the dataset
to be used for evaluating the model during the training
# Model training returns model history(accuracy, loss, epochs...)

history = model_1.fit(X_train_scaled, y_train, epochs=60,
validation_split=0.15)
```

```
Epoch 1/60
13/13 [==============================] - 1s 35ms/step - loss: 0.7219 -
accuracy: 0.3705 - val_loss: 0.7088 - val_accuracy: 0.4928
Epoch 2/60
13/13 [==============================] - 0s 5ms/step - loss: 0.7113 -
accuracy: 0.4793 - val_loss: 0.7006 - val_accuracy: 0.5362
Epoch 3/60
13/13 [==============================] - 0s 5ms/step - loss: 0.7019 -
accuracy: 0.5259 - val_loss: 0.6937 - val_accuracy: 0.5362
Epoch 4/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6942 -
accuracy: 0.5855 - val_loss: 0.6871 - val_accuracy: 0.5942
Epoch 5/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6871 -
accuracy: 0.6606 - val_loss: 0.6801 - val_accuracy: 0.7101
Epoch 6/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6798 -
accuracy: 0.7202 - val_loss: 0.6735 - val_accuracy: 0.7536
Epoch 7/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6728 -
accuracy: 0.7565 - val_loss: 0.6675 - val_accuracy: 0.7971
Epoch 8/60
13/13 [==============================] - 0s 6ms/step - loss: 0.6662 -
```

```
accuracy: 0.7798 - val_loss: 0.6610 - val_accuracy: 0.7971
Epoch 9/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6595 -
accuracy: 0.8187 - val_loss: 0.6549 - val_accuracy: 0.7391
Epoch 10/60
13/13 [==============================] - 0s 7ms/step - loss: 0.6536 -
accuracy: 0.7694 - val_loss: 0.6492 - val_accuracy: 0.7391
Epoch 11/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6479 -
accuracy: 0.7642 - val_loss: 0.6439 - val_accuracy: 0.7246
Epoch 12/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6423 -
accuracy: 0.7409 - val_loss: 0.6388 - val_accuracy: 0.7246
Epoch 13/60
13/13 [==============================] - 0s 6ms/step - loss: 0.6371 -
accuracy: 0.7306 - val_loss: 0.6344 - val_accuracy: 0.7826
Epoch 14/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6318 -
accuracy: 0.8057 - val_loss: 0.6294 - val_accuracy: 0.8116
Epoch 15/60
13/13 [==============================] - 0s 6ms/step - loss: 0.6264 -
accuracy: 0.8238 - val_loss: 0.6245 - val_accuracy: 0.8406
Epoch 16/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6211 -
accuracy: 0.8446 - val_loss: 0.6194 - val_accuracy: 0.8116
Epoch 17/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6156 -
accuracy: 0.8472 - val_loss: 0.6142 - val_accuracy: 0.7971
Epoch 18/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6103 -
accuracy: 0.8446 - val_loss: 0.6090 - val_accuracy: 0.8406
Epoch 19/60
13/13 [==============================] - 0s 5ms/step - loss: 0.6044 -
accuracy: 0.8653 - val_loss: 0.6038 - val_accuracy: 0.8696
Epoch 20/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5986 -
accuracy: 0.8964 - val_loss: 0.5988 - val_accuracy: 0.8696
Epoch 21/60
13/13 [==============================] - 0s 6ms/step - loss: 0.5931 -
accuracy: 0.9016 - val_loss: 0.5928 - val_accuracy: 0.8696
Epoch 22/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5869 -
accuracy: 0.9093 - val_loss: 0.5874 - val_accuracy: 0.8696
Epoch 23/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5811 -
accuracy: 0.9145 - val_loss: 0.5814 - val_accuracy: 0.8841
Epoch 24/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5749 -
accuracy: 0.9145 - val_loss: 0.5749 - val_accuracy: 0.8841
```

```
Epoch 25/60
13/13 [==============================] - 0s 6ms/step - loss: 0.5678 -
accuracy: 0.9197 - val_loss: 0.5684 - val_accuracy: 0.8696
Epoch 26/60
13/13 [==============================] - 0s 6ms/step - loss: 0.5612 -
accuracy: 0.9119 - val_loss: 0.5618 - val_accuracy: 0.8696
Epoch 27/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5545 -
accuracy: 0.9119 - val_loss: 0.5553 - val_accuracy: 0.8696
Epoch 28/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5477 -
accuracy: 0.9119 - val_loss: 0.5486 - val_accuracy: 0.8696
Epoch 29/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5406 -
accuracy: 0.9145 - val_loss: 0.5413 - val_accuracy: 0.8696
Epoch 30/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5331 -
accuracy: 0.9171 - val_loss: 0.5348 - val_accuracy: 0.8841
Epoch 31/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5262 -
accuracy: 0.9197 - val_loss: 0.5280 - val_accuracy: 0.8986
Epoch 32/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5194 -
accuracy: 0.9275 - val_loss: 0.5208 - val_accuracy: 0.8986
Epoch 33/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5116 -
accuracy: 0.9275 - val_loss: 0.5138 - val_accuracy: 0.8986
Epoch 34/60
13/13 [==============================] - 0s 5ms/step - loss: 0.5042 -
accuracy: 0.9378 - val_loss: 0.5068 - val_accuracy: 0.8841
Epoch 35/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4970 -
accuracy: 0.9197 - val_loss: 0.4999 - val_accuracy: 0.8986
Epoch 36/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4897 -
accuracy: 0.9326 - val_loss: 0.4928 - val_accuracy: 0.8841
Epoch 37/60
13/13 [==============================] - 0s 6ms/step - loss: 0.4825 -
accuracy: 0.9275 - val_loss: 0.4852 - val_accuracy: 0.8841
Epoch 38/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4749 -
accuracy: 0.9326 - val_loss: 0.4781 - val_accuracy: 0.8841
Epoch 39/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4676 -
accuracy: 0.9249 - val_loss: 0.4714 - val_accuracy: 0.8841
Epoch 40/60
13/13 [==============================] - 0s 6ms/step - loss: 0.4605 -
accuracy: 0.9223 - val_loss: 0.4642 - val_accuracy: 0.8841
Epoch 41/60
```

```
13/13 [==============================] - 0s 5ms/step - loss: 0.4533 -
accuracy: 0.9223 - val_loss: 0.4572 - val_accuracy: 0.8841
Epoch 42/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4463 -
accuracy: 0.9197 - val_loss: 0.4501 - val_accuracy: 0.8841
Epoch 43/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4394 -
accuracy: 0.9223 - val_loss: 0.4420 - val_accuracy: 0.8986
Epoch 44/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4314 -
accuracy: 0.9404 - val_loss: 0.4351 - val_accuracy: 0.8841
Epoch 45/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4244 -
accuracy: 0.9352 - val_loss: 0.4273 - val_accuracy: 0.9130
Epoch 46/60
13/13 [==============================] - 0s 6ms/step - loss: 0.4172 -
accuracy: 0.9352 - val_loss: 0.4203 - val_accuracy: 0.9130
Epoch 47/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4102 -
accuracy: 0.9326 - val_loss: 0.4135 - val_accuracy: 0.9420
Epoch 48/60
13/13 [==============================] - 0s 5ms/step - loss: 0.4037 -
accuracy: 0.9326 - val_loss: 0.4062 - val_accuracy: 0.9130
Epoch 49/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3964 -
accuracy: 0.9378 - val_loss: 0.3993 - val_accuracy: 0.9130
Epoch 50/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3896 -
accuracy: 0.9378 - val_loss: 0.3925 - val_accuracy: 0.9130
Epoch 51/60
13/13 [==============================] - 0s 6ms/step - loss: 0.3829 -
accuracy: 0.9352 - val_loss: 0.3861 - val_accuracy: 0.9275
Epoch 52/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3767 -
accuracy: 0.9326 - val_loss: 0.3797 - val_accuracy: 0.9130
Epoch 53/60
13/13 [==============================] - 0s 6ms/step - loss: 0.3703 -
accuracy: 0.9404 - val_loss: 0.3740 - val_accuracy: 0.9275
Epoch 54/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3649 -
accuracy: 0.9301 - val_loss: 0.3675 - val_accuracy: 0.9420
Epoch 55/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3586 -
accuracy: 0.9326 - val_loss: 0.3614 - val_accuracy: 0.9130
Epoch 56/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3520 -
accuracy: 0.9378 - val_loss: 0.3558 - val_accuracy: 0.9420
Epoch 57/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3470 -
```

```
accuracy: 0.9352 - val_loss: 0.3497 - val_accuracy: 0.9420
Epoch 58/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3408 -
accuracy: 0.9326 - val_loss: 0.3441 - val_accuracy: 0.9130
Epoch 59/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3352 -
accuracy: 0.9378 - val_loss: 0.3407 - val_accuracy: 0.8986
Epoch 60/60
13/13 [==============================] - 0s 5ms/step - loss: 0.3304 -
accuracy: 0.9404 - val_loss: 0.3338 - val_accuracy: 0.9275
```

I trained for 60 epochs. That was quick.

Let's visualize accuracy and loss to actually see how the model did. It is always easy to notice performance on graph than looking on models training progress above.

!! If you retrain again, it will continue where it left. So, for example, if you train for 30 epochs, and you rerun the cell, it will train for same more epochs again.

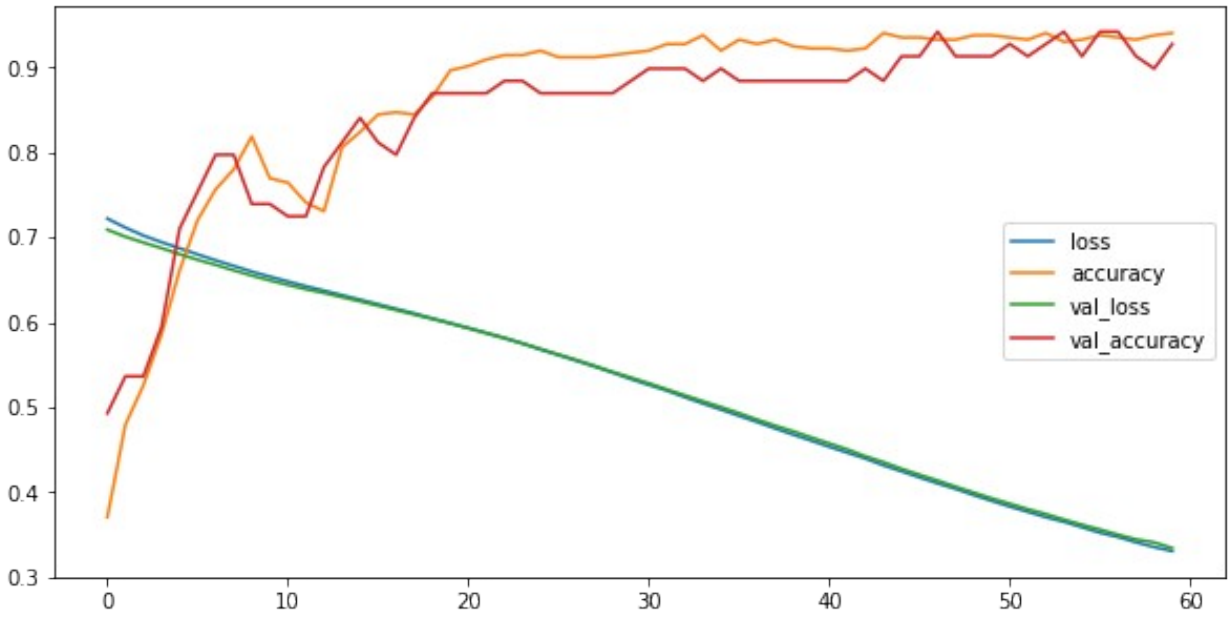
2.5 Visualizing the Results

Visualizing the model results after training is always a good way to learn what you can do to improve the performance.

Let's get a Pandas dataframe containing training loss and accuracy, and validation loss and accuracy.

```
# Getting the dataframe of loss and accuracies on both training and
validation

loss_acc_metrics_df = pd.DataFrame(history.history)
loss_acc_metrics_df.plot(figsize=(10,5))
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f2d80451f90>
```



This is really impressive. Seems that for only 60 epochs, the training accuracy was up to 90% while validation accuracy was 84%.

This is not bad considering that we have only 455 training samples, and also 15% of such samples are allocated to the validation set. The validation accuracy can be increased by increaing the validation samples.

Let's evaluate the model on the test set.

2.6 Evaluating the Model

Quite often, you will want to test your model on the data that it never saw. This data is normally called `test set` and in more applied practice, you will only feed the test to the model after you have done your best to improve it.

Let's now evaluate the model on the test set. One thing to note here is that the test set must be preprocessed the same way we preprocessed the training set. The training set was rescaled (with `MinMaxScaler`) and that was applied to the test set.

If this is not obeyed, you would not know why you're having poor results. Just look up on the next next cell how poor the accuracy will be if I evaluate the model on unscaled data when I trained it on scaled data.

```
# Evaluating a model on unseen data: test set

model_eval = model_1.evaluate(X_test_scaled, y_test)

# Printing the loss and accuracy
```

```
print('Test loss: {}\nTest accuracy:
{}'.format(model_eval[0],model_eval[1]))
```

```
4/4 [==============================] - 0s 4ms/step - loss: 0.3146 -
accuracy: 0.9298
Test loss: 0.3145657181739807
Test accuracy:0.9298245906829834
```

```
# !!DON'T DO THIS!! X_test is not scaled. The results will be awful

model_1.evaluate(X_test, y_test)
```

```
4/4 [==============================] - 0s 4ms/step - loss: 327.3092 -
accuracy: 0.3772

[327.3092346191406, 0.37719297409057617]
```

It's very suprising how the model did on the test data. It achieved 93%.

Accuracy is one classification metric, but there are more metrics such as f1 score, recall, and precision. The easiest way to get these metrics is to use `classification_report` function provided by Scikit-Learn.

Sometime you would also want to know how your model did on the both positive and negative examples. In this case you can use confusion matrix to see the predicted and the actual classes.

```
# Getting the prediction

predictions = model_1.predict(X_test_scaled)

predictions[:15]
```

```
array([[0.68703896],
       [0.19852844],
       [0.37471318],
       [0.78504544],
       [0.8081874 ],
       [0.03121866],
       [0.04509011],
       [0.37985504],
       [0.48416775],
       [0.73546237],
       [0.75935453],
       [0.40290105],
       [0.71097237],
       [0.5158887 ],
       [0.7241085 ]], dtype=float32)
```

If you look at the predictions above, they are probabilities (value between 0 and 1).

And this make sense because our model is returning the values between 0 and 1 because of the `sigmoid activation function or logistic function` at the output layer.

In order to find the metrics we noted above, we have to round the predictions to either 0 or 1. For this, we can use `np.round()` or `tf.math.round()`.

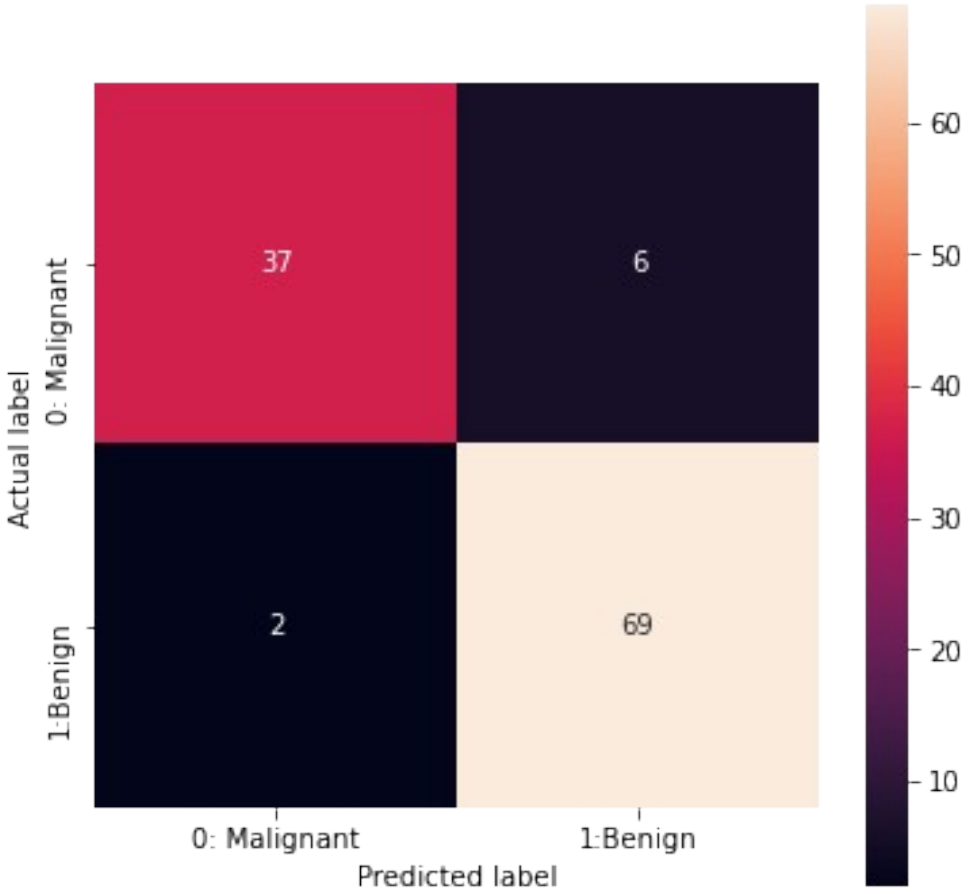
Round function will return the closest integer. For example, for a prediction of 0.3, it will be 0. If the prediction is 0.6, the closest integer is 1.

```
# Rounding the predictions to 0 and 1

predictions = tf.round(predictions)

 # Display the first 15 preds values

predictions[:15]
```

```
<tf.Tensor: shape=(15, 1), dtype=float32, numpy=
array([[1.],
       [0.],
       [0.],
       [1.],
       [1.],
       [0.],
       [0.],
       [0.],
       [0.],
       [1.],
       [1.],
       [0.],
       [1.],
       [1.],
       [1.]], dtype=float32)>
```

Great, the predictions are now rounded up to either 0 or 1.

```
# Getting the confusion matrix

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, predictions)

# Plotting confusion matrix

plt.figure(figsize=(6,6))
sns.heatmap(cm, square=True, annot=True, fmt='d', cbar=True,
                        xticklabels=['0: Malignant', '1:Benign'],
                        yticklabels=['0: Malignant', '1:Benign'])
plt.ylabel('Actual label')
plt.xlabel('Predicted label');
```



Let's interpret the confusion matrix:

First off, the rows represent the actual classes and the columns represent the predicted classes.

With that said:

- *36 Malignant samples were correctly classified as Malignant. Also called True Positives*
- *7 Malignant samples were incorrectly classified as Benign. Also called False Positives*
- *0 Benign samples were incorrectly classified as Malignant. Also called False Negatives*
- *71 Benign samples were correctly classified as Benign. Also called True Negatives*

```
# Classification report: F1 score, Recall, Precision

from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.95      0.86      0.90        43
           1       0.92      0.97      0.95        71

    accuracy                           0.93       114
```

```
   macro avg       0.93      0.92      0.92       114
weighted avg       0.93      0.93      0.93       114
```

Here are notes about these metrics:

- **Accuracy** is the ratio of the correct predicted samples over the total samples.
- **Precision** is the ratio of correct predicted positive samples over the total positive predictions.
- **Recall** is the ration of correct predicted positive samples over total positive samples.
- **F1 score** is the harmonic mean of precision and recall.

These metrics can be confusing. To learn more about them, here is a great writeup by Santiago..

Going Beyond Binary Classifier to Multiclass Classifier: 10 Fashions Classifier

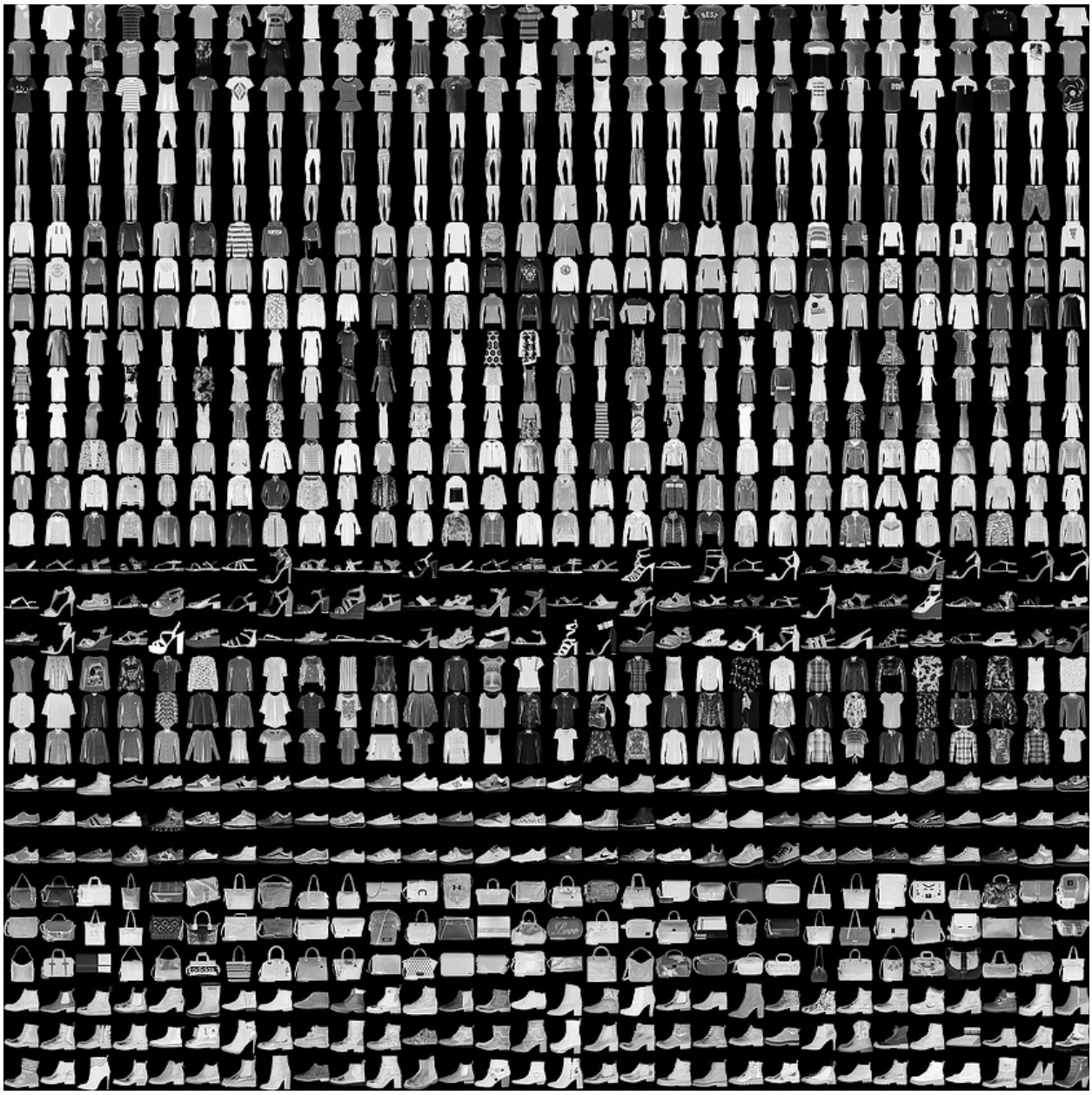
So far, we have built a neural network for regression(in previous labs) and binary classification. And we have only been working with structured datasets(datasets in tabular format).

Can the same neural networks we used be able to recognize images? In this next practice, we will turn the page to image classification. We will build a neural network for recognizing 10 different fashions and along the way, we will learn other things such as stopping the training upon a given condition is met, and using TensorBoard to visualize model.

That is going to be cool! Let's get started!

3.1 Getting the Fashion data

Fashion MNIST data is made of 70.000 fashions, 60.000 of them are allocated for training set and 10.000 for test set. Each image is 28*28, grayscale.



The fashions are of 10 categories. Below are their labels:

Label	Description
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Image & gif of fashions shown above are taken from dataset homepage

Let's get the dataset from Keras.

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist
```

```
(fashion_train, fashion_train_label), (fashion_test,
fashion_test_label) = fashion_mnist.load_data()
```

Looking in the Fashion Data

As always, it is a best practice to peep into the images to see how they like.

Let's display the pixels values of a given image, image, and its corresponding label.

```
index = 10

# Get the pixels

fashion_train[index]
```

```
array([[  0,   0,   0,   0,   0,   0,   0,  11, 142, 200, 106,   0,
0,
          0,   0,   0,   0,   0,  85, 185, 112,   0,   0,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0, 152, 214, 217, 194, 236, 216,
187,
        149, 135, 153, 211, 217, 231, 205, 217, 188,  34,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,  66, 185, 166, 180, 181, 190, 211,
221,
        197, 146, 198, 206, 191, 168, 190, 172, 188, 175,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0, 135, 153, 160, 175, 180, 170, 186,
187,
        190, 188, 190, 187, 174, 195, 185, 174, 161, 175,  59,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0, 161, 147, 160, 170, 178, 177, 180,
168,
        173, 174, 171, 185, 184, 185, 172, 171, 164, 174, 120,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   2, 175, 146, 145, 168, 178, 181, 185,
180,
        184, 178, 179, 187, 191, 193, 190, 181, 171, 172, 158,   0,
0,
          0,   0],
       [  0,   0,   0,   0,  35, 177, 155, 140, 151, 172, 191, 187,
186,
```

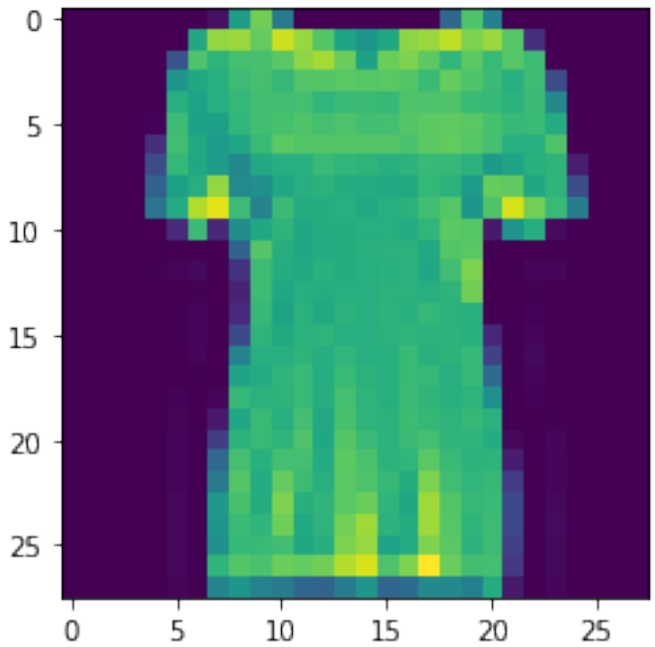
```
        187, 186, 187, 182, 191, 194, 188, 180, 161, 161, 185,   0,
0,
          0,   0],
       [  0,   0,   0,   0,  59, 170, 153, 141, 120, 154, 160, 161,
172,
        168, 166, 161, 165, 172, 170, 164, 139, 149, 162, 166,  21,
0,
          0,   0],
       [  0,   0,   0,   0,  79, 145, 160, 214, 123, 128, 153, 160,
164,
        158, 157, 154, 155, 170, 165, 141, 195, 193, 152, 166,  61,
0,
          0,   0],
       [  0,   0,   0,   0, 100, 157, 225, 245, 175, 113, 174, 158,
158,
        160, 155, 160, 164, 178, 188, 135, 185, 240, 201, 172, 108,
0,
          0,   0],
       [  0,   0,   0,   0,   0,  31, 174,  28, 126, 153, 166, 152,
158,
        158, 160, 161, 157, 168, 191, 188,  18, 132, 159,   7,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,  82, 187, 159, 153,
157,
        158, 162, 164, 164, 154, 187, 190,   0,   0,   0,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   1,   3,   5,   0,  37, 175, 158, 155,
162,
        158, 160, 162, 165, 153, 177, 205,   0,   0,   3,   3,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   1,   0,  25, 175, 152, 160,
158,
        161, 160, 164, 164, 161, 166, 200,   0,   0,   1,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   4,   0,  30, 171, 147, 164,
155,
        165, 161, 165, 162, 170, 164, 162,   0,   0,   2,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   4,   0,  57, 166, 155, 164,
166,
        161, 161, 164, 167, 165, 165, 162,  28,   0,   3,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   3,   0, 114, 161, 161, 166,
```

```
159,
        168, 161, 161, 172, 162, 165, 171,  50,   0,   5,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   1,   0, 149, 157, 167, 172,
159,
        172, 164, 161, 172, 170, 160, 171,  89,   0,   4,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   2,   0,   4, 171, 164, 166, 173,
159,
        179, 166, 160, 174, 167, 162, 166, 128,   0,   2,   0,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   3,   0,  18, 152, 173, 160, 179,
154,
        181, 166, 164, 175, 170, 166, 170, 164,   0,   0,   1,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   4,   0,  47, 165, 172, 167, 185,
153,
        187, 173, 165, 174, 179, 166, 166, 158,   5,   0,   3,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   4,   0,  87, 180, 162, 179, 179,
157,
        191, 182, 165, 168, 190, 173, 165, 166,  20,   0,   4,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   4,   0, 105, 187, 157, 194, 175,
161,
        190, 184, 170, 158, 205, 177, 168, 171,  44,   0,   4,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   5,   0, 138, 181, 158, 205, 160,
167,
        190, 198, 167, 152, 218, 186, 170, 172,  57,   0,   5,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   5,   0, 135, 174, 167, 199, 155,
166,
        201, 219, 165, 158, 218, 188, 167, 175,  56,   0,   7,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   5,   0, 129, 171, 172, 177, 153,
159,
        206, 216, 148, 157, 206, 190, 165, 175,  48,   0,   5,   0,
0,
          0,   0],
```

```
       [  0,   0,   0,   0,   0,   5,   0, 167, 187, 182, 198, 194,
200,
        226, 240, 184, 206, 255, 197, 178, 179,  42,   0,   5,   0,
0,
          0,   0],
       [  0,   0,   0,   0,   0,   3,   0, 115, 135, 113, 106,  85,
82,
        108, 133,  83,  90, 121, 120, 110, 158,  18,   0,   3,   0,
0,
          0,   0]], dtype=uint8)
```

```
# A list of label names

class_names = ['T-shirt/top',
'Trouser','Pullover','Dress','Coat','Sandal','Shirt','Sneaker','Bag','
Ankle boot']

# Show the image

plt.imshow(fashion_train[index])

# Display the label

image_label = fashion_train_label[index]
print('This type of fashion is: {}
({})'.format(class_names[image_label], image_label))
```

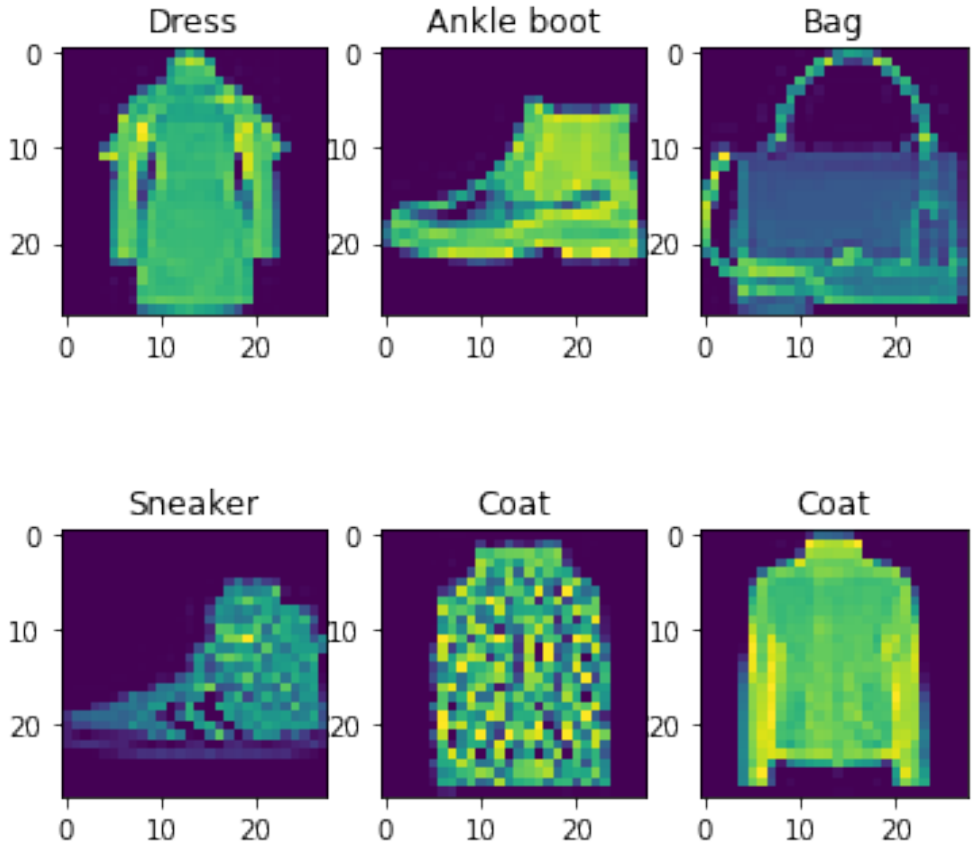
```
This type of fashion is: T-shirt/top(0)
```



The fashions with the label 0 is `T-shirt/top`. Normally, the pixels of image range from 0 to 255. If you can look back where we displayed the pixels, you will see that they vary from 0 to 255.

We can also visualize some random images.

```
import random

plt.figure(figsize=(6,6))

for index in range(6):

  ax = plt.subplot(2,3, index+1)
  random_index = random.choice(range(len(fashion_train)))
  plt.imshow(fashion_train[random_index])
  plt.title(class_names[fashion_train_label[random_index]])
```



You can rerun the above cell to display different fashions.

Another important thing to look at when working with images is to see their size.

This is important because later when we will be creating a model, we have to specify the input shape and such shape is same as the shape of the images. Each image is 28*28, but let's verify that.

```
# Getting the image shape

print('The shape of the whole training dataset:
{}'.format(fashion_train[0].shape))
print('The shape of the first(and other)image:
{}'.format(fashion_train[0].shape))
```

```
The shape of the whole training dataset:(28, 28)
The shape of the first(and other)image:(28, 28)
```

Now that we know the dataset that we are working with, let us do some few proprocessing before building a model.

3.3 Preparing the Data

In many cases, real world images datasets are not that clean like fashion mnist.

You may have to correct images that were incorrectly labeled, or you have labels in texts that need to be converted to numbers(most machine learning models accept numeric input), or scale the pixels values.

The latter is what we are going to do. It is inarguable that scaling the images pixels to value between 0 and 1 increase the performance of the neural network, and hence the results. Let's do it!!

As we have seen, the pixels range from 0 to 255. So we will divide both training and test set by 255.0.

```
# Scaling the image pixels to be between 0 and 1

fashion_train = fashion_train/255.0

fashion_test = fashion_test/255.0
```

We are now ready to build a neural network.

3.4 Creating, Compiling, and Training a Model

There are few points to note before creating a model:

- When working with images, the shape of the input images has to be correctly provided. This is a common error done by many people, including me (before I learned it).

- This is a multiclass classification, which is different to the binary classifier we did earlier. The difference will be reflected in the choice of output activation function, number of output neurons, and the loss function.
- That said, we will use `softmax` as activation in the last layer, 10 neurons or units because we have 10 fashions, and the loss will be `SparseCategoricalCrossentropy` because the labels are pure integer. If the labels were in one hot format, we would use `CategoricalCrossentropy`. Learn more about Keras losses here.

Documentation is always the top source when learning all the possibilities of any framework. And also, Keras doc is beautifuly organized. Not to mention that the keras API is also well designed as well.

Let's now create a model.

```
# Creating a model

fashion_classifier = tf.keras.models.Sequential([

        # Flattening layer will convert array of pixels into one
dimensional column array
       tf.keras.layers.Flatten(),
       tf.keras.layers.Dense(units=64, activation='relu'),
       tf.keras.layers.Dense(units=32, activation='relu'),
       tf.keras.layers.Dense(units=10, activation='softmax')


])

# Compiling a model: Specifying a loss and optimization function

fashion_classifier.compile(optimizer='adam',
                           loss='sparse_categorical_crossentropy',
                           metrics=['accuracy']

)
```

Now that we built and compiled a model, we can train it.

In order to train a model, we must have an input data and output(labels). We train the model to get the relationship between the input and output. Such relationship is what we tend to call rules. So, in other words, we provide the data and the answers to a model to get the rules.

```
# Training a model
# Allocating 15% of training data to validation set

fashion_classifier.fit(fashion_train, fashion_train_label, epochs=20,
validation_split=0.15)
```

```
Epoch 1/20
1594/1594 [==============================] - 6s 3ms/step - loss:
0.5368 - accuracy: 0.8105 - val_loss: 0.4270 - val_accuracy: 0.8456
Epoch 2/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3994 - accuracy: 0.8576 - val_loss: 0.3924 - val_accuracy: 0.8610
Epoch 3/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3631 - accuracy: 0.8678 - val_loss: 0.3694 - val_accuracy: 0.8654
Epoch 4/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3387 - accuracy: 0.8765 - val_loss: 0.3942 - val_accuracy: 0.8583
Epoch 5/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3190 - accuracy: 0.8827 - val_loss: 0.3612 - val_accuracy: 0.8720
Epoch 6/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3068 - accuracy: 0.8861 - val_loss: 0.3365 - val_accuracy: 0.8770
Epoch 7/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2916 - accuracy: 0.8922 - val_loss: 0.3551 - val_accuracy: 0.8750
Epoch 8/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2814 - accuracy: 0.8968 - val_loss: 0.3471 - val_accuracy: 0.8734
Epoch 9/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2728 - accuracy: 0.8985 - val_loss: 0.3460 - val_accuracy: 0.8763
Epoch 10/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2634 - accuracy: 0.9019 - val_loss: 0.3611 - val_accuracy: 0.8781
Epoch 11/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2559 - accuracy: 0.9047 - val_loss: 0.3387 - val_accuracy: 0.8834
Epoch 12/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2484 - accuracy: 0.9065 - val_loss: 0.3395 - val_accuracy: 0.8810
Epoch 13/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2441 - accuracy: 0.9081 - val_loss: 0.3416 - val_accuracy: 0.8826
Epoch 14/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2352 - accuracy: 0.9124 - val_loss: 0.3782 - val_accuracy: 0.8686
Epoch 15/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2313 - accuracy: 0.9144 - val_loss: 0.3559 - val_accuracy: 0.8828
Epoch 16/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2262 - accuracy: 0.9157 - val_loss: 0.3359 - val_accuracy: 0.8887
Epoch 17/20
1594/1594 [==============================] - 5s 3ms/step - loss:
```

```
0.2212 - accuracy: 0.9177 - val_loss: 0.3734 - val_accuracy: 0.8754
Epoch 18/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2166 - accuracy: 0.9190 - val_loss: 0.3660 - val_accuracy: 0.8803
Epoch 19/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2098 - accuracy: 0.9226 - val_loss: 0.3611 - val_accuracy: 0.8831
Epoch 20/20
1594/1594 [==============================] - 6s 3ms/step - loss:
0.2091 - accuracy: 0.9207 - val_loss: 0.3630 - val_accuracy: 0.8802

<keras.callbacks.History at 0x7f2d48291310>
```

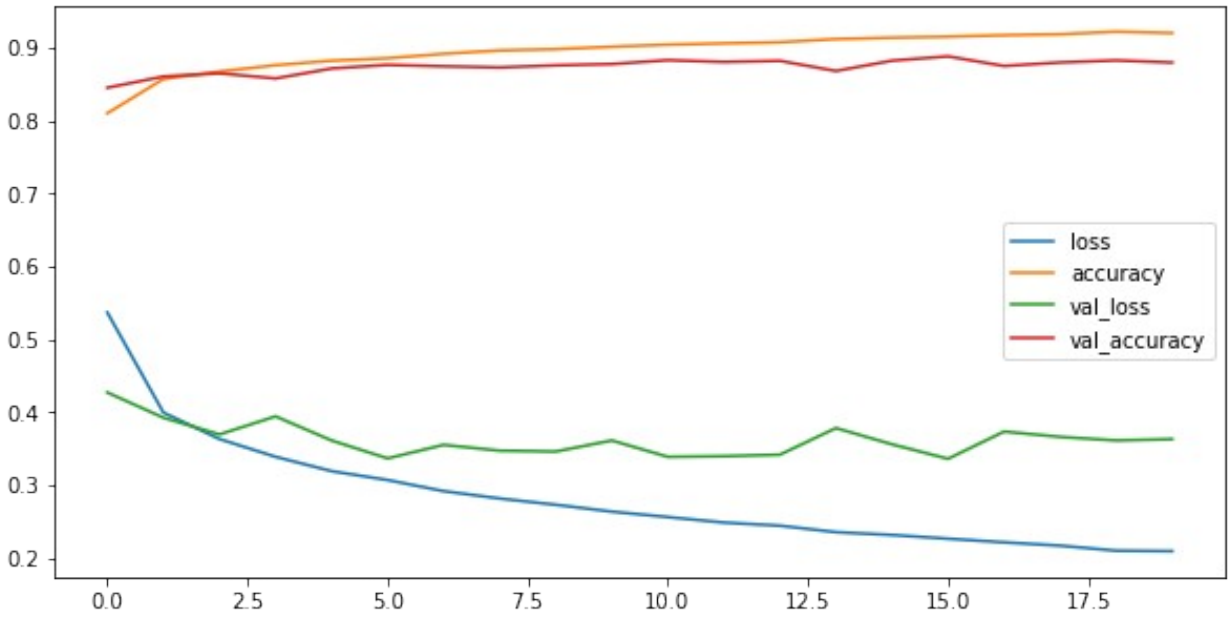
This was fast. When using Google Colab, you can speed up the training by changing the `runtime type`to `GPU`. You can head over `Runtime` in the `menu bar`>> Click on `Change runtime type`>>Choose `GPU`.

But also, training mnist for 20 epochs is not slow that we would need to activate GPU. We will take an advantage of GPU in later labs.

3.5 Visualizing the Model Results

Let's visualize the model results to see how training went.

```
# Getting the dataframe of loss and accuracies on both training and
validation

loss_acc_metrics_df = pd.DataFrame(fashion_classifier.history.history)

# Plotting the loss and accuracy

loss_acc_metrics_df.plot(figsize=(10,5))

<matplotlib.axes._subplots.AxesSubplot at 0x7f2d482ef810>
```



At the end of the training, the accuracy is about 92% while validation accuracy being 88% or so. That's not bad considering that we built a simple model and trained for only 20 epochs.

Let's see how the model performs on unseed data: test set.

3.6 Model Evaluation

```
# Evaluating the model on unseen data

eval = fashion_classifier.evaluate(fashion_test, fashion_test_label)

# Printing the loss and accuracy

print('Test loss: {}\nTest accuracy:{}'.format(eval[0],eval[1]))
```

```
313/313 [==============================] - 1s 2ms/step - loss: 0.3724
- accuracy: 0.8799
Test loss: 0.3724398612976074
Test accuracy:0.8798999786376953
```

The fashion classifier that we built is 88% confident at recognizing unseen fashion.

We could also find other Classification metrics based on True/False positives & negatives such as precision, recall, but since we already saw how to compute them in the binary classifier, let's see other interesting things: Controlling the training using callbacks and using TensorBoard.

3.7 Controlling Training with Callbacks

We can use Callbacks functions to control the training.

Take an example: we can stop training when the model is lo longer showing significant improvements on validation set. Or we can terminate training when a certain condition is met.

Implementing Callbacks

There are various functionalities available in Keras Callbacks.

Let's start with how to use `ModelCheckpoint` to save the model when the performance on the validation set is best so far. By saving the best model on the validation set, we avoid things like overfitting which is a common issue in machine learning model training, neural network specifically. We also train for less time.

I will rebuild a same model again.

```
# Creating a same model as used before

def classifier():

  model = tf.keras.models.Sequential([

        # Flattening layer will convert array of pixels into one
dimensional column array
       tf.keras.layers.Flatten(),
       tf.keras.layers.Dense(units=64, activation='relu'),
       tf.keras.layers.Dense(units=32, activation='relu'),
       tf.keras.layers.Dense(units=10, activation='softmax')


  ])

# Compiling a model: Specifying a loss and optimization function

  model.compile(optimizer='adam',
                           loss='sparse_categorical_crossentropy',
                           metrics=['accuracy']

  )

  return model

# Defining callbacks

from keras.callbacks import ModelCheckpoint

callbacks = ModelCheckpoint('fashion_classifier.h5',
save_best_only=True)
```

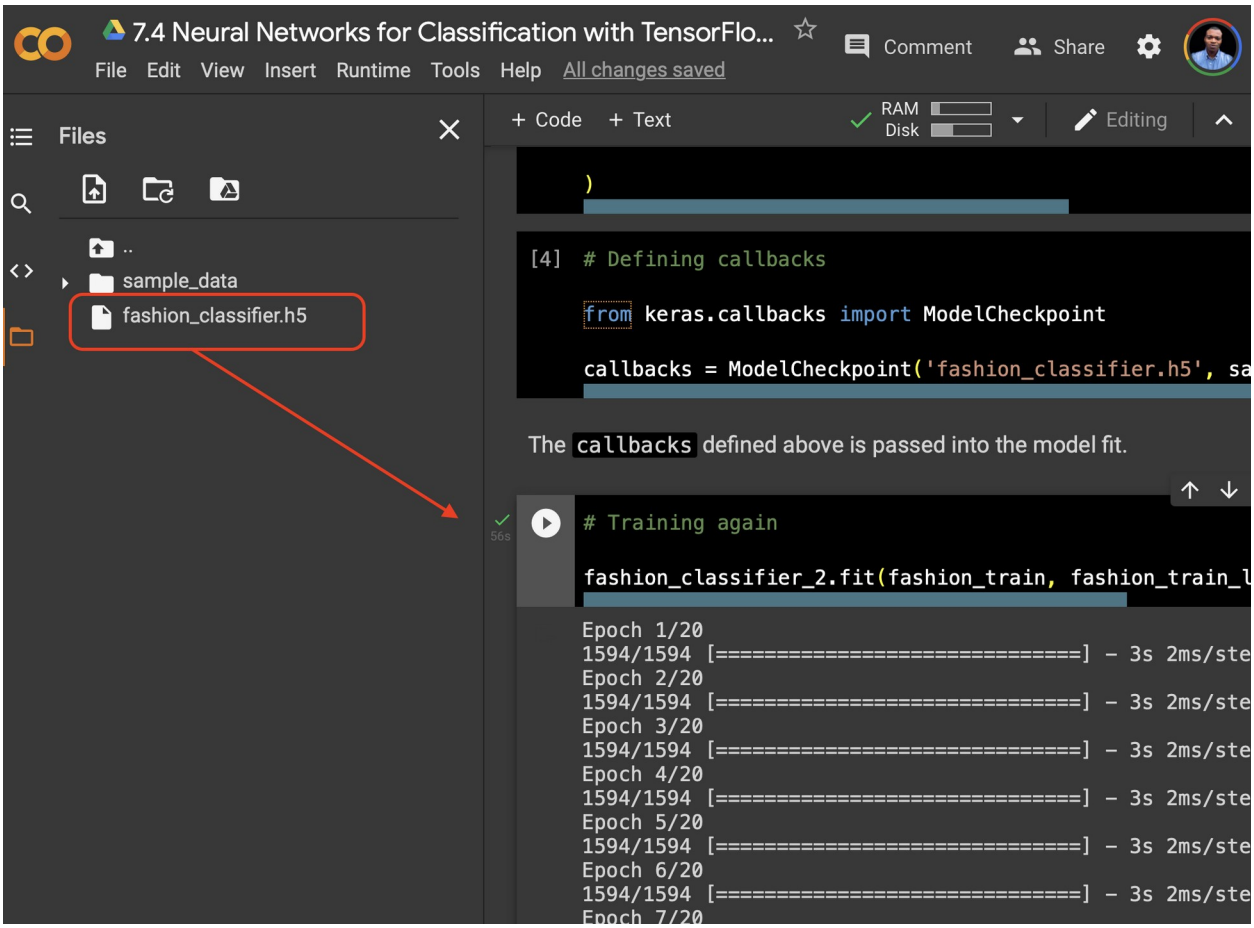
The `callbacks` defined above is passed into the model fit.

```
# Controlling training with callbacks

# Get the model

fashion_classifier_2 = classifier()

fashion_classifier_2.fit(fashion_train, fashion_train_label,
epochs=20, validation_split=0.15, callbacks=[callbacks])
```

```
Epoch 1/20
1594/1594 [==============================] - 6s 3ms/step - loss:
0.5474 - accuracy: 0.8101 - val_loss: 0.4271 - val_accuracy: 0.8476
Epoch 2/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.4015 - accuracy: 0.8573 - val_loss: 0.4191 - val_accuracy: 0.8470
Epoch 3/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3643 - accuracy: 0.8689 - val_loss: 0.3805 - val_accuracy: 0.8650
Epoch 4/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3396 - accuracy: 0.8761 - val_loss: 0.3639 - val_accuracy: 0.8667
Epoch 5/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3177 - accuracy: 0.8834 - val_loss: 0.3656 - val_accuracy: 0.8678
Epoch 6/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3048 - accuracy: 0.8870 - val_loss: 0.3419 - val_accuracy: 0.8746
Epoch 7/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2910 - accuracy: 0.8931 - val_loss: 0.3551 - val_accuracy: 0.8748
Epoch 8/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2818 - accuracy: 0.8949 - val_loss: 0.3306 - val_accuracy: 0.8814
Epoch 9/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2706 - accuracy: 0.8986 - val_loss: 0.3336 - val_accuracy: 0.8832
Epoch 10/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2635 - accuracy: 0.9021 - val_loss: 0.3203 - val_accuracy: 0.8878
Epoch 11/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2554 - accuracy: 0.9044 - val_loss: 0.3499 - val_accuracy: 0.8786
Epoch 12/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2476 - accuracy: 0.9061 - val_loss: 0.3305 - val_accuracy: 0.8842
Epoch 13/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2410 - accuracy: 0.9091 - val_loss: 0.3473 - val_accuracy: 0.8809
```

```
Epoch 14/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2354 - accuracy: 0.9117 - val_loss: 0.3207 - val_accuracy: 0.8858
Epoch 15/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2288 - accuracy: 0.9138 - val_loss: 0.3310 - val_accuracy: 0.8876
Epoch 16/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2249 - accuracy: 0.9165 - val_loss: 0.3339 - val_accuracy: 0.8866
Epoch 17/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2222 - accuracy: 0.9159 - val_loss: 0.3273 - val_accuracy: 0.8896
Epoch 18/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2163 - accuracy: 0.9184 - val_loss: 0.3223 - val_accuracy: 0.8923
Epoch 19/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2103 - accuracy: 0.9214 - val_loss: 0.3735 - val_accuracy: 0.8820
Epoch 20/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2072 - accuracy: 0.9212 - val_loss: 0.3380 - val_accuracy: 0.8882

<keras.callbacks.History at 0x7f2d480378d0>
```

If you look below, the model was saved and the version that is saved will be the best model on the validation set.



Another easier way to control the model training is to use `EarlyStopping`.

By using early stopping, the training will be stopped when a monitored metric is no longer improving for a given number of consecutive epochs.

The metric to be monitored during the training is `val_accuracy` in our example, and it is assigned to `monitor` argument. The `patience` represent the number of consecutive epochs of which the training will stop when there is no significant improvements in `val_accuracy`.

```
from keras.callbacks import EarlyStopping

early_stop = EarlyStopping(monitor='val_accuracy', patience=4,
restore_best_weights=True)
```

Let's train for 100 epochs to be able to notice if the training stops as soon as there is no improvements in the validation set for 4 epochs in row.

```
# Stopping training early

# Getting the model

fashion_classifier_2 = classifier()
```

```
history = fashion_classifier_2.fit(fashion_train, fashion_train_label,
epochs=100, validation_split=0.15, callbacks=[early_stop])
```

```
Epoch 1/100
1594/1594 [==============================] - 6s 3ms/step - loss:
0.5298 - accuracy: 0.8124 - val_loss: 0.4781 - val_accuracy: 0.8349
Epoch 2/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3938 - accuracy: 0.8577 - val_loss: 0.3845 - val_accuracy: 0.8653
Epoch 3/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3545 - accuracy: 0.8701 - val_loss: 0.3672 - val_accuracy: 0.8672
Epoch 4/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3294 - accuracy: 0.8788 - val_loss: 0.3474 - val_accuracy: 0.8734
Epoch 5/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3124 - accuracy: 0.8864 - val_loss: 0.3533 - val_accuracy: 0.8702
Epoch 6/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2955 - accuracy: 0.8904 - val_loss: 0.3438 - val_accuracy: 0.8770
Epoch 7/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2847 - accuracy: 0.8945 - val_loss: 0.3271 - val_accuracy: 0.8862
Epoch 8/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2748 - accuracy: 0.8987 - val_loss: 0.3329 - val_accuracy: 0.8836
Epoch 9/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2690 - accuracy: 0.8993 - val_loss: 0.3317 - val_accuracy: 0.8847
Epoch 10/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2597 - accuracy: 0.9030 - val_loss: 0.3388 - val_accuracy: 0.8803
Epoch 11/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2522 - accuracy: 0.9065 - val_loss: 0.3447 - val_accuracy: 0.8813
```

The training stopped at the epoch of 12, with the accuracy of 88.33%. On epoch 8, the accuracy was 88.48, it is very clear that from the epoch 8 to 12, there was no improvement, and thus the training stopped.

Early stopping can potentially save our time and resources.

One last thing before going to custom callbacks, we can combine both `ModelCheckpoint`(for saving the best model on validation set on the course of training) and `EarlyStopping` for saving us time.

```
# Combining Early stopping and Model Check point
```

```
# Getting the model

fashion_classifier_2 = classifier()

history = fashion_classifier_2.fit(fashion_train, fashion_train_label,
epochs=100, validation_split=0.15, callbacks=[callbacks, early_stop])

Epoch 1/100
1594/1594 [==============================] - 6s 3ms/step - loss:
0.5287 - accuracy: 0.8138 - val_loss: 0.4567 - val_accuracy: 0.8323
Epoch 2/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3910 - accuracy: 0.8588 - val_loss: 0.3919 - val_accuracy: 0.8586
Epoch 3/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3532 - accuracy: 0.8701 - val_loss: 0.3811 - val_accuracy: 0.8614
Epoch 4/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3329 - accuracy: 0.8783 - val_loss: 0.3745 - val_accuracy: 0.8633
Epoch 5/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3132 - accuracy: 0.8853 - val_loss: 0.3593 - val_accuracy: 0.8720
Epoch 6/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3026 - accuracy: 0.8882 - val_loss: 0.3393 - val_accuracy: 0.8771
Epoch 7/100
1594/1594 [==============================] - 6s 3ms/step - loss:
0.2864 - accuracy: 0.8947 - val_loss: 0.3387 - val_accuracy: 0.8808
Epoch 8/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2807 - accuracy: 0.8966 - val_loss: 0.3348 - val_accuracy: 0.8842
Epoch 9/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2688 - accuracy: 0.8997 - val_loss: 0.3263 - val_accuracy: 0.8868
Epoch 10/100
1594/1594 [==============================] - 6s 3ms/step - loss:
0.2606 - accuracy: 0.9030 - val_loss: 0.3465 - val_accuracy: 0.8799
Epoch 11/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2528 - accuracy: 0.9059 - val_loss: 0.3243 - val_accuracy: 0.8844
Epoch 12/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2460 - accuracy: 0.9076 - val_loss: 0.3345 - val_accuracy: 0.8872
Epoch 13/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2392 - accuracy: 0.9098 - val_loss: 0.3339 - val_accuracy: 0.8899
Epoch 14/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2324 - accuracy: 0.9116 - val_loss: 0.3322 - val_accuracy: 0.8861
Epoch 15/100
```

```
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2269 - accuracy: 0.9150 - val_loss: 0.3262 - val_accuracy: 0.8898
Epoch 16/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2214 - accuracy: 0.9161 - val_loss: 0.3421 - val_accuracy: 0.8843
Epoch 17/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2157 - accuracy: 0.9190 - val_loss: 0.3452 - val_accuracy: 0.8887
```

Custom Callback

Keras offers various functions for implementing custom callbacks that are very handy when you want to control the model training with a little bit of customization.

You can do certain actions on almost every step of the training. Let's stop the training when the accuracy is 95%.

```
# Custom callbacks

class callback(tf.keras.callbacks.Callback):

  def on_epoch_end(self, epoch, logs={}):

    if (logs.get('accuracy') > 0.95):

      print('\n Training is cancelled at an accuracy of 95%')
      self.model.stop_training = True


# Call callbacks

custom_callback = callback()

# Implementing custom ballback

# Getting the model

fashion_classifier_2 = classifier()

history = fashion_classifier_2.fit(fashion_train, fashion_train_label,
epochs=100, validation_split=0.15, callbacks=[custom_callback])
```

```
Epoch 1/100
1594/1594 [==============================] - 6s 3ms/step - loss:
0.5393 - accuracy: 0.8095 - val_loss: 0.4304 - val_accuracy: 0.8489
Epoch 2/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3875 - accuracy: 0.8606 - val_loss: 0.4043 - val_accuracy: 0.8553
Epoch 3/100
1594/1594 [==============================] - 5s 3ms/step - loss:
```

```
0.3518 - accuracy: 0.8713 - val_loss: 0.3831 - val_accuracy: 0.8613
Epoch 4/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3307 - accuracy: 0.8785 - val_loss: 0.3949 - val_accuracy: 0.8592
Epoch 5/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3134 - accuracy: 0.8834 - val_loss: 0.3427 - val_accuracy: 0.8756
Epoch 6/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2982 - accuracy: 0.8904 - val_loss: 0.3421 - val_accuracy: 0.8746
Epoch 7/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2865 - accuracy: 0.8937 - val_loss: 0.3488 - val_accuracy: 0.8768
Epoch 8/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2787 - accuracy: 0.8961 - val_loss: 0.3465 - val_accuracy: 0.8731
Epoch 9/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2679 - accuracy: 0.9009 - val_loss: 0.3399 - val_accuracy: 0.8794
Epoch 10/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2627 - accuracy: 0.9026 - val_loss: 0.3354 - val_accuracy: 0.8771
Epoch 11/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2515 - accuracy: 0.9060 - val_loss: 0.3654 - val_accuracy: 0.8758
Epoch 12/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2490 - accuracy: 0.9078 - val_loss: 0.3355 - val_accuracy: 0.8858
Epoch 13/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2407 - accuracy: 0.9099 - val_loss: 0.3607 - val_accuracy: 0.8772
Epoch 14/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2352 - accuracy: 0.9106 - val_loss: 0.3417 - val_accuracy: 0.8816
Epoch 15/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2308 - accuracy: 0.9141 - val_loss: 0.3394 - val_accuracy: 0.8841
Epoch 16/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2243 - accuracy: 0.9169 - val_loss: 0.3393 - val_accuracy: 0.8871
Epoch 17/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2210 - accuracy: 0.9147 - val_loss: 0.3471 - val_accuracy: 0.8849
Epoch 18/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2131 - accuracy: 0.9192 - val_loss: 0.3469 - val_accuracy: 0.8817
Epoch 19/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2109 - accuracy: 0.9211 - val_loss: 0.3470 - val_accuracy: 0.8842
```

```
Epoch 20/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2077 - accuracy: 0.9211 - val_loss: 0.3504 - val_accuracy: 0.8873
Epoch 21/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2009 - accuracy: 0.9247 - val_loss: 0.3628 - val_accuracy: 0.8866
Epoch 22/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1975 - accuracy: 0.9264 - val_loss: 0.3652 - val_accuracy: 0.8866
Epoch 23/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1953 - accuracy: 0.9271 - val_loss: 0.3722 - val_accuracy: 0.8846
Epoch 24/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1889 - accuracy: 0.9287 - val_loss: 0.3790 - val_accuracy: 0.8836
Epoch 25/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1890 - accuracy: 0.9296 - val_loss: 0.3630 - val_accuracy: 0.8882
Epoch 26/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1842 - accuracy: 0.9303 - val_loss: 0.3635 - val_accuracy: 0.8866
Epoch 27/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1813 - accuracy: 0.9308 - val_loss: 0.3846 - val_accuracy: 0.8817
Epoch 28/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1802 - accuracy: 0.9313 - val_loss: 0.3738 - val_accuracy: 0.8856
Epoch 29/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1744 - accuracy: 0.9335 - val_loss: 0.3925 - val_accuracy: 0.8803
Epoch 30/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1717 - accuracy: 0.9347 - val_loss: 0.3876 - val_accuracy: 0.8857
Epoch 31/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1681 - accuracy: 0.9373 - val_loss: 0.3768 - val_accuracy: 0.8896
Epoch 32/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1677 - accuracy: 0.9373 - val_loss: 0.4369 - val_accuracy: 0.8799
Epoch 33/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1651 - accuracy: 0.9377 - val_loss: 0.3976 - val_accuracy: 0.8868
Epoch 34/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1628 - accuracy: 0.9388 - val_loss: 0.4188 - val_accuracy: 0.8846
Epoch 35/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1585 - accuracy: 0.9400 - val_loss: 0.4172 - val_accuracy: 0.8842
Epoch 36/100
```

```
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1565 - accuracy: 0.9413 - val_loss: 0.4243 - val_accuracy: 0.8809
Epoch 37/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1541 - accuracy: 0.9423 - val_loss: 0.4241 - val_accuracy: 0.8858
Epoch 38/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1530 - accuracy: 0.9425 - val_loss: 0.4433 - val_accuracy: 0.8846
Epoch 39/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1512 - accuracy: 0.9435 - val_loss: 0.4563 - val_accuracy: 0.8827
Epoch 40/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1468 - accuracy: 0.9452 - val_loss: 0.4485 - val_accuracy: 0.8834
Epoch 41/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1444 - accuracy: 0.9458 - val_loss: 0.4425 - val_accuracy: 0.8836
Epoch 42/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1420 - accuracy: 0.9458 - val_loss: 0.4627 - val_accuracy: 0.8843
Epoch 43/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1412 - accuracy: 0.9468 - val_loss: 0.4497 - val_accuracy: 0.8882
Epoch 44/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1392 - accuracy: 0.9478 - val_loss: 0.4436 - val_accuracy: 0.8903
Epoch 45/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1380 - accuracy: 0.9484 - val_loss: 0.4693 - val_accuracy: 0.8878
Epoch 46/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1364 - accuracy: 0.9491 - val_loss: 0.4452 - val_accuracy: 0.8882
Epoch 47/100
1594/1594 [==============================] - 5s 3ms/step - loss:
0.1354 - accuracy: 0.9502 - val_loss: 0.4672 - val_accuracy: 0.8849

 Training is cancelled at an accuracy of 95%
```

Perfect! The training stopped as soon as the training accuracy reached 95% and the specified message was printed.

There are many customizations available in Keras custom callbacks. Be sure to check that out.

3.8 Using TensorBoard for Model Visualization

Tensorboard is incredible tool used by many people (and not just only TensorFlow developers) to experiment with machine learning.

With TensorBoard, you can:

- Track and visualize the loss and accuracy
- Visualize the model graphs and operations
- Display images and other types of data
- View the histograms of weights and biases.

We first have to load the TensorBoard extension as follows.

```
# Load the Tensorboard notebook extension
# And import datetime

%load_ext tensorboard
```

And we clear all logs from all runs we did before.

```
!rm -rf ./logs/
```

Le's get the model from the function `classifier` defined in previous cells.

```
# Getting the model

fashion_classifier = classifier()
```

Let's create a Keras callback.

```
# Create a callback

tfboard_callback = tf.keras.callbacks.TensorBoard(log_dir="logs")
```

Now, train the model and provide the `tfboard_callback` to `callback` argument.

```
fashion_classifier.fit(fashion_train, fashion_train_label, epochs=20,
validation_split=0.15, callbacks=[tfboard_callback])

Epoch 1/20
1594/1594 [==============================] - 6s 4ms/step - loss:
0.5468 - accuracy: 0.8092 - val_loss: 0.4526 - val_accuracy: 0.8351
Epoch 2/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3985 - accuracy: 0.8584 - val_loss: 0.3857 - val_accuracy: 0.8616
Epoch 3/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3600 - accuracy: 0.8689 - val_loss: 0.3876 - val_accuracy: 0.8622
Epoch 4/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3338 - accuracy: 0.8776 - val_loss: 0.3388 - val_accuracy: 0.8774
Epoch 5/20
1594/1594 [==============================] - 5s 3ms/step - loss:
```

```
0.3169 - accuracy: 0.8836 - val_loss: 0.3621 - val_accuracy: 0.8713
Epoch 6/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.3049 - accuracy: 0.8876 - val_loss: 0.3392 - val_accuracy: 0.8787
Epoch 7/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2918 - accuracy: 0.8925 - val_loss: 0.3668 - val_accuracy: 0.8717
Epoch 8/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2799 - accuracy: 0.8972 - val_loss: 0.3521 - val_accuracy: 0.8762
Epoch 9/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2731 - accuracy: 0.8988 - val_loss: 0.3483 - val_accuracy: 0.8762
Epoch 10/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2643 - accuracy: 0.9018 - val_loss: 0.3473 - val_accuracy: 0.8807
Epoch 11/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2570 - accuracy: 0.9045 - val_loss: 0.3285 - val_accuracy: 0.8822
Epoch 12/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2474 - accuracy: 0.9092 - val_loss: 0.3551 - val_accuracy: 0.8760
Epoch 13/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2432 - accuracy: 0.9099 - val_loss: 0.3375 - val_accuracy: 0.8880
Epoch 14/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2374 - accuracy: 0.9112 - val_loss: 0.3658 - val_accuracy: 0.8696
Epoch 15/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2306 - accuracy: 0.9142 - val_loss: 0.3499 - val_accuracy: 0.8806
Epoch 16/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2267 - accuracy: 0.9158 - val_loss: 0.3433 - val_accuracy: 0.8833
Epoch 17/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2200 - accuracy: 0.9181 - val_loss: 0.3512 - val_accuracy: 0.8832
Epoch 18/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2157 - accuracy: 0.9200 - val_loss: 0.3501 - val_accuracy: 0.8844
Epoch 19/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2105 - accuracy: 0.9219 - val_loss: 0.3604 - val_accuracy: 0.8833
Epoch 20/20
1594/1594 [==============================] - 5s 3ms/step - loss:
0.2072 - accuracy: 0.9219 - val_loss: 0.3500 - val_accuracy: 0.8842

<keras.callbacks.History at 0x7f2d44992110>
```

Now that the training is over, we can start the TensorBoard.

```
%tensorboard --logdir logs
```

```
<IPython.core.display.Javascript object>
```

As you can see, TensorBoard is very useful. The fact that you can use it to visualize the performance metrics, model graphs, and datasets as well.

3.9 Final Notes

This is the end of the notebook!

We have learned how to build neural networks for binary classification, multiclass classification, saw how to control the taining with callbacks, and how to use TensorBoard for visualizing model, metrics, and parameters.

BACK TO TOP

```
```